



Node.js Important Topics:

This sheet provides a quick reference to key Node.js concepts. Keep practicing and exploring deeper topics!

1. Node.js Basics

- **Node.js:** Node.js is a **runtime environment** built on **V8 JavaScript engine** that allows you to run JavaScript on the server-side. It's designed for building scalable network applications using an **event-driven, non-blocking I/O model**.
- **Single-threaded** but uses an event-driven, non-blocking architecture.
- **Libuv Library:** A **C** library that powers Node.js with an event loop and thread pool for asynchronous c operations.

2. Modules and Exports

- **CommonJS** (`require`, `module.exports`): Used for modular programming.
- `require()` is a function used to import modules, into your Node.js application.
- **ES Modules** (`import`, `export default`): Modern approach to modules.
- **Built-in Modules:** `fs`, `path`, `http`, `crypto`, `os`, etc.
- **Middleware:** Functions in Express.js that execute between request and response cycles.

3. Event Loop & Concurrency

- **Event Loop:** Handles asynchronous tasks using the libuv library.
- **Phases:** Timers, I/O callbacks, idle & prepare, poll, check, close callbacks.
- **Concurrency Handling:**
 - `setImmediate()`, `process.nextTick()`, `setTimeout()`.
 - Worker Threads & Clustering.

4. [Express.js](#)

- **Web Framework:** Simplifies building REST APIs.

- **Middlewares:** Functions that handle requests before reaching routes
- A function that processes the request/response before it reaches the final route or after it leaves it..
- **Routing:** `app.get()`, `app.post()`, `app.put()`, `app.delete()`.

5. REST APIs & HTTP Methods

Http is the standard protocol used for transmitted data over web. data transmitted is not encrypted,

REST-An architecture style that uses HTTP methods to perform CRUD operations

- **GET:** Read data.
- **POST:** Create data
- **PUT:** Update entire resource.
- **PATCH:** Update partial resource.
- **DELETE:** Remove data.
- **GraphQL:** An alternative to REST for querying APIs with more flexibility.

6. Security & Authentication

- **JWT (JSON Web Token):** Secure token-based authentication.
- **Token Expiry:** Set using the `expiresIn` property in JWT.
- **Login Flow:** User submits credentials -> Server verifies -> JWT token issued -> Token stored and sent for authentication.
- **OAuth:** Thirdk, GitHub, etc.). Open Authorization. It enables third party applications to interact with APIs. Like github.
- **Helmet.js:** Enhances security by setting various HTTP headers.
- **Hashing & Salting:** Using `bcrypt` for pass-party login (Google, Facebook words).
- **CORS (Cross-Origin Resource Sharing):** Controls API access from different origins.

7. Database (SQL & NoSQL)

- **Primary Key:** Unique identifier for a record.
- **Foreign Key:** Links two tables together.
- **Joins:** INNER, LEFT, RIGHT, FULL.
- **DELETE vs TRUNCATE:** DELETE removes specific records, TRUNCATE removes all records but keeps table structure.
- **MongoDB:** NoSQL database using BSON. (Binary JSON)

8. File System & Streams+++++++++

- **fs module:** Read/write file operations.

- **Streams:** Streams are objects that handle reading or writing data in **chunks** (as opposed to loading the entire data in memory at once). There are four types of streams: Readable, Writable, Duplex, Transform.
- **Buffers:** buffer is a temporary storage area for chunks of [data](#) . it is a container A **Buffer** in Node.js is a **binary data structure** that stores data in memory. It allows you to work with raw binary data, like reading or writing files, network requests, etc.. Handle binary data.
- **Streams** are used to handle large amounts of data efficiently in chunks.
- **Buffer** is used to work with binary data.

9. Process Management

- **PM2:** Process manager for [Node.js](#).
 - `pm2 start app.js -u` Starts the app.
 - `pm2 list` - Lists running applications.
 - `pm2 restart app` - Restarts the application.
- **NODE_ENV:** Environment variables for different stages (dev, staging, production).
In Node.js, the **NODE_ENV** environment variable is **used to define the environment** in which your application is running
- **Clustering & Forking:** Enhances performance by utilizing multiple CPU cores.
 - **Fork:** Creates a new process.
 - **Spawn:** Runs a command in a new process
- **.Load Balancing:** Distributes incoming requests among multiple servers to improve performance.

10.Process Management

- **PM2:** Process manager for [Node.js](#).
 - `pm2 start app.js -u` Starts the app.
 -
 - `pm2 list` - Lists running applications.
 - `pm2 restart app` - Restarts the application.
- **NODE_ENV:** Environment variables for different stages (dev, staging, production).
In Node.js, the **NODE_ENV** environment variable is **used to define the environment** in which your application is running
- **Clustering & Forking:** Enhances performance by utilizing multiple CPU cores.
 - **Fork:** Creates a new process.
 - **Spawn:** Runs a command in a new process
- **.Load Balancing:** Distributes incoming requests among multiple servers to improve performance.

11. Testing in Node.js

- **Jest:** Popular testing framework for JavaScript. Maintained by Meta(Facebook)
- **TDD (Test-Driven Development):** It is a **software development approach** where **you write tests before writing the actual code**.
- **Unit Testing:** Testing individual functions or components.
- **Integration Testing:** Testing multiple components together.

12. Deployment & Scaling

- **AWS, Docker, Kubernetes:** Deploying Node.js apps.
- **AWS Deployment Steps:**
 1. Push code to GitHub.
 2. SSH into AWS EC2 instance
 3. Pull the latest code.
 4. Install dependencies.
 5. Start the server using PM2.
- **Load Balancing:** Distributes traffic across multiple instances.
- **Scaling with Clusters:** Uses **cluster** modules for multi-core processing.

13. CI/CD Pipelines

- **Continuous Integration (CI):** Automated testing and integration of code.
- **Continuous Deployment (CD):** Automated deployment to production.
- **Tools:** Jenkins, GitHub Actions, GitLab CI/CD, CircleCI.

14. .gitignore Best Practices

1. **Exclude sensitive files:**
 - node_modules/
 2. .env
 - 3.
 4. logs/
- package-lock.json

.gitignore is a special file used in Git to **exclude files/folders** from being tracked or pushed to the repository.

15. Advanced Concepts

- **Reactor & Proactor Patterns:** Used for handling asynchronous I/O operations in Node.js. Node.js (via **libuv**) uses **design patterns** like **Reactor** and **Proactor** to handle non-blocking I/O.

- **Microservices:** Architectural style for developing applications as small, independent services.
 - **Serverless Architecture:** Running code without managing servers using AWS Lambda, Firebase Functions, etc.
-

HOISTING & LET, VAR, CONST

- **Execution Context:** The environment in which JS code runs. Created in two phases: memory allocation & code execution.
 - **Hoisting:** Declarations are moved to the top. `var` is hoisted with `undefined`, `let` and `const` are hoisted but not initialized will be in the temporal dead zone.
 - **Temporal Dead Zone:** The time between hoisting and initialization where accessing `let/const` throws an reference error.
 - **Difference:**
 - `var`: function-scoped, hoisted.
 - `let`: block-scoped, not re-declarable, hoisted but TDZ.
 - `const`: block-scoped, not reassignable.
-

CLOSURE

- **Closure:** A function remembers variables from its outer scope even after the outer function has finished.
 - **Used in:** Data privacy, currying, event handlers, etc.
-

CALLBACK

- **Callback:** Function passed to another function as an argument.
 - **Callback Hell:** Nested callbacks making code hard to read.
 - **Solution:** Use Promises or `async/await`.
-

PROMISE

- **Promise:** Represents a value which may be available now, or in the future.
 - **Methods:**
 - `.all([ ])`: Resolves when all promises resolve.
 - `.any([ ])`: Resolves when first fulfills.
 - `.race([ ])`: Resolves/rejects when first settles.
-

ASYNCHRONOUS JS

- **API:** Interface to fetch data (like HTTP endpoints).
- **Axios Methods:**
 - `GET`: fetch data.
 - `POST`: send new data.
 - `PUT`: replace data.
 - `PATCH`: update partial data.
 - `DELETE`: remove data.
- **`async/await`:** Makes promises easier to work with; still returns a promise.

- **Limitations:** Can't be used in top-level code in some environments.
 - **Event Loop:** Handles async tasks via queues.
 - **Call Stack:** Executes synchronous code.
 - **Callback Queue:** For `setTimeout`, DOM events.
-
- **Microtask Queue:** For promises.

CALL, APPLY, BIND

- **Call:** `fun.call(thisArg, arg1, arg2)` — calls with this & args.
- Call function which is use to invoke a function directly by passing in the reference which point to this variable inside a function .
- **Apply:** `fun.apply(thisArg, [args])` — same as call, but args in array.
Same as call function the only difference is that It take a argument in array list
- **Bind:** `fun.bind(thisArg)` — returns new function with bound context.
- **Use Cases:** Borrow methods, set `this`, function currying.

ARRAY METHODS

- `.map()`: transforms each element.
- `.filter()`: filters elements by condition.
- `.reduce()`: reduces array to a value.
- `.forEach()`: iterates, no return.

- `.find()`: returns first matching element.
-

```
let arr=[  
  {fname:"abu",lname:"umar",age:24},  
  {fname:"billu",lname:"man",age:31},  
  {fname:"gull",lname:"kumar",age:18},  
  {fname:"zain",lname:"khan",age:12},  
]  
  
let second=arr.filter(x=>{  
  return x.age>20  
}).map(x=>{  
  return x.fname  
})
```

`console.log(second)//output [abu,billu]`

We can change as well

```
let arr=[5,1,3,2,6]  
  
let output=arr.reduce((acc,curr)=>{  
  let max=0  
  if(curr>max)  
  {  
    max=curr  
  }  
  return curr
```

```
})  
  
console.log(output)
```

OBJECT METHODS

- `Object.keys()`: returns property names.
 - `Object.values()`: returns property values.
 - `Object.entries()`: key-value pairs array.
-

STRING METHODS

- `.replace()`: replaces first match.
 - `.replaceAll()`: replaces all matches.
 - `.toLowerCase()` / `.toUpperCase()`: changes case.
 - `.split()`: splits string to array.
 - `.slice()`: extracts part.
 - `.splice()`: (Array method, not string) modifies array.
-

OOPs IN JS

- **Pillars**: Encapsulation, Abstraction, Inheritance, Polymorphism.
- **Class**: Template for creating objects.
- **Constructor**: Method to initialize.

- **Static:** Belongs to class, not instances.
 - static method is used to implement function that belong to a class as a whole not to particular object.
 - **Super:** Refers to parent class.
 - **Prototype:** JS objects inherit from other objects.
 - **Prototype Chain:** Inheritance chain of objects.
 - "In JavaScript, every object has a prototype. If we try to access a property that doesn't exist on the object, JavaScript checks its prototype, and so on — this is called the **prototype chain**. Using this, we can achieve inheritance, where one object gets properties or methods from another object. This is known as **prototype inheritance**."
-

FUNCTIONS

- **Higher Order Function:** Accepts/returns another function.
 - **Callback Function:** Passed into another function.
 - **First Class Functions:** Functions treated like variables.
 - **Arrow Functions:** Shorter syntax, no own `this`.
-

SPREAD & REST

- **Spread (...arr):** Expands arrays/objects. Merge two array, clone means copy like original array
- `const numbers = [1, 2, 3];`
- `function add(a, b, c) {`
- `return a + b + c;`
- `}`
- `console.log(add(...numbers)); // Output: 6`
-
- **Rest (...args):** use in parameter, Collects multiple args into one. destructuring
function add(...b)

- {
- let sum=0
- console.log(b)
- for(let i of b)
- {
- sum=sum+i
- }
- console.log(sum)
- return sum
-
- }
- console.log(add(4,5,3))
- //////////output
- [4, 5, 3]
- 12
- 12

the rest operator collects elements into a single entity (an array or object), while the spread operator expands an entity into its individual elements or properties.

CURRYING

- **Currying:** Currying is a function that *takes one argument at a time and returns a new function expecting the next argument.*
- **Use Case:** Partial application.
- **Infinite Currying:** (a) => (b) => (c) => ...
- //////////////////////////////////
- function add(a)
- {
- return function(b)
- {
- if(b) return add(a+b)
- return a
- }
- }
- }
- console.log(add(2)(1)(4)(3)())



BROWSER STORAGE

- **LocalStorage:** Persistent storage, key-value, sync. 10 mb max size
- **SessionStorage:** Clears on tab close. 5 mb size
- **Cookies:** Small data sent with every HTTP request. 4 kb max size
- **Common Status Codes:**
 - 200: OK
 - 201: Created
 - 400: Bad Request
 - 500: Server Error